

UNIVERSIDAD ADOLFO IBÁÑEZ

Programa MIA

SISTEMA MULTI-AGÉNTICO CON OPENAI API

Pipeline de 5 Agentes: Del Intent al Supervisor
Aplicado al Universo Star Wars

Curso: Simulaciones Basadas en Agentes

Profesor: Ahmad Armoush

Año 2026

1. Introducción

Este documento presenta el diseño, arquitectura e implementación de un sistema multi-agéntico construido en Python utilizando la API de OpenAI. El sistema implementa un pipeline secuencial de cinco agentes especializados, cada uno con un rol diferenciado, que colaboran para procesar consultas de usuario de manera robusta y fiscalizada.

El caso de uso principal se enmarca en el universo Star Wars, donde los agentes clasifican intenciones, buscan información canónica, validan consistencia, generan respuestas estructuradas y fiscalizan la calidad final antes de entregarla al usuario.

Este proyecto se desarrolla en el contexto del curso Simulaciones Basadas en Agentes del Programa MIA de la Universidad Adolfo Ibáñez, bajo la dirección del profesor Ahmad Armoush.

2. Arquitectura General del Pipeline

El sistema sigue una arquitectura de pipeline secuencial donde cada agente recibe el output del agente anterior, procesa su tarea específica y pasa el resultado al siguiente. Un Orchestrator central coordina la ejecución, recopila trazas y maneja errores.

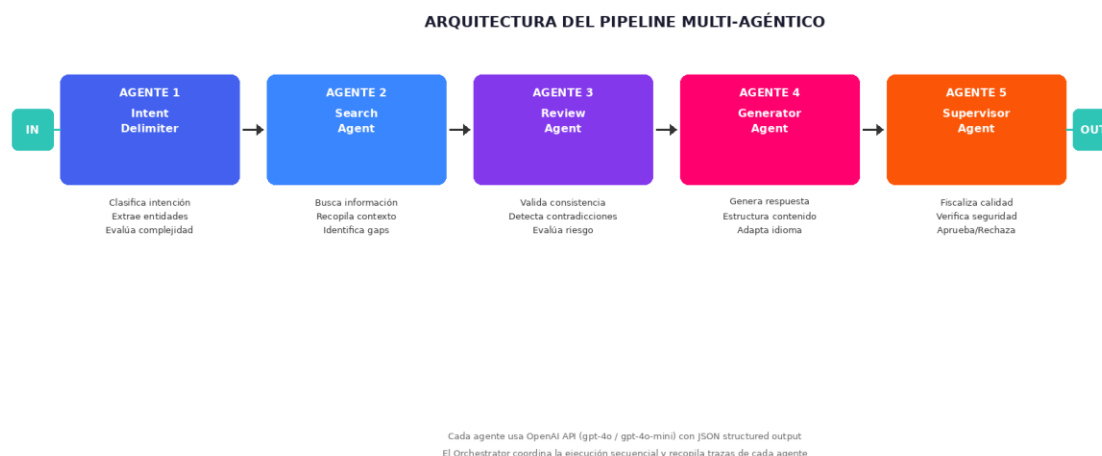


Figura 1: Arquitectura del pipeline multi-agéntico con 5 agentes especializados

La comunicación entre agentes es unidireccional (izquierda a derecha). Cada agente produce un JSON estructurado que se acumula en el payload del pipeline. El Supervisor tiene la autoridad final para aprobar, modificar o rechazar la respuesta generada.

3. Descripción Detallada de los Agentes

3.1 Agente 1: Intent Delimiter (Delimitador de Intención)

Este es el primer agente del pipeline y actúa como puerta de entrada. Su responsabilidad es analizar el mensaje del usuario y clasificar la intención, extraer entidades relevantes, detectar el idioma y evaluar la complejidad de la consulta.

Propiedad	Detalle
Modelo	gpt-4o-mini (optimizado para velocidad)
Output	category, sub_intent, entities, parameters, language, complexity, confidence
Categorías	question, action, analysis, creative, clarification, unknown
Ejemplo SW	Query: ¿Quién es Darth Vader? → category: question, entities: [{character: Darth Vader}]

3.2 Agente 2: Search Agent (Agente de Búsqueda)

Recibe la intención clasificada y busca información relevante para construir la respuesta. Genera resultados estructurados con scores de relevancia e identifica gaps de conocimiento que podrían afectar la calidad de la respuesta.

Propiedad	Detalle
Modelo	gpt-4o (requiere razonamiento complejo)
Output	search_results (source, content, relevance_score), knowledge_gaps, suggested_queries
Ejemplo SW	Busca: canon Star Wars sobre Darth Vader → relevance_score: 0.95

3.3 Agente 3: Review Agent (Agente Revisor)

Actúa como control de calidad intermedio. Examina los resultados de búsqueda verificando consistencia interna, detectando contradicciones entre fuentes, evaluando completitud y clasificando el nivel de riesgo de la información recopilada.

Propiedad	Detalle
Modelo	gpt-4o

Output	is_consistent, consistency_score, issues, contradictions, risk_level, validated_facts
Niveles riesgo	low (datos canónicos), medium (interpretaciones), high (info no verificada)

3.4 Agente 4: Generator Agent (Agente Generador)

Sintetiza toda la información recopilada y validada para generar la respuesta final. Considera la intención clasificada, los hechos validados, las advertencias del revisor y el contexto del usuario para producir una respuesta completa, estructurada y en el idioma correcto.

Propiedad	Detalle
Modelo	gpt-4o
Output	response, format, summary, key_points, caveats, confidence
Ejemplo SW	Genera respuesta completa sobre Darth Vader con hechos validados y caveats

3.5 Agente 5: Supervisor Agent (Agente Fiscalizador)

Es la última línea de defensa antes de entregar la respuesta al usuario. Evalúa cinco dimensiones: completitud, precisión, relevancia, seguridad y tono. Tiene la autoridad para aprobar, modificar o rechazar la respuesta según umbrales definidos.

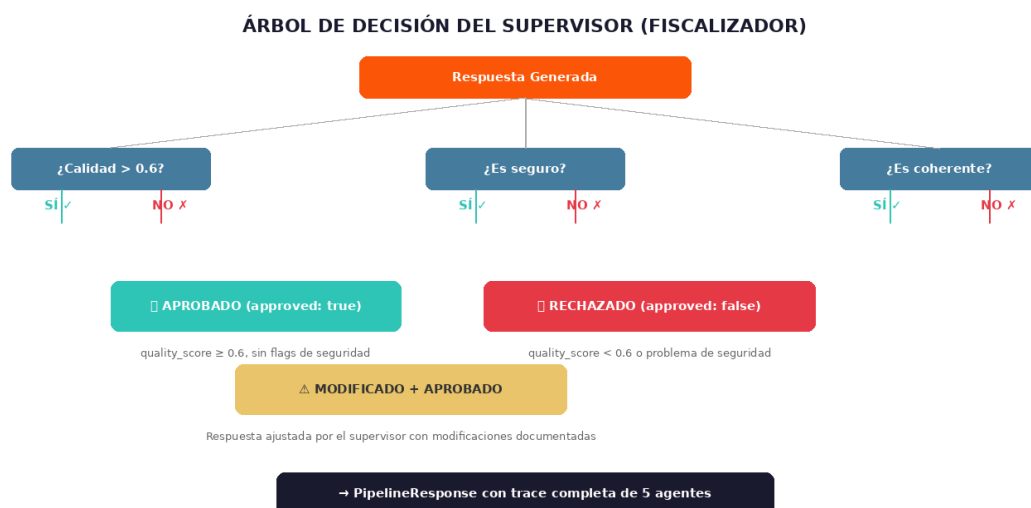


Figura 2: Árbol de decisión del agente Supervisor/Fiscalizador

Propiedad	Detalle
Modelo	gpt-4o
Output	approved, quality_score, flags, category_checks, modifications, final_response
Umbral rechazo	quality_score < 0.6 o cualquier flag de seguridad → RECHAZADO

4. Flujo de Datos Completo

El siguiente diagrama muestra el flujo completo de datos a través del pipeline, usando como ejemplo la query: ¿Quién es Darth Vader? Cada agente recibe el output acumulado de los agentes anteriores y agrega su propio resultado al payload.

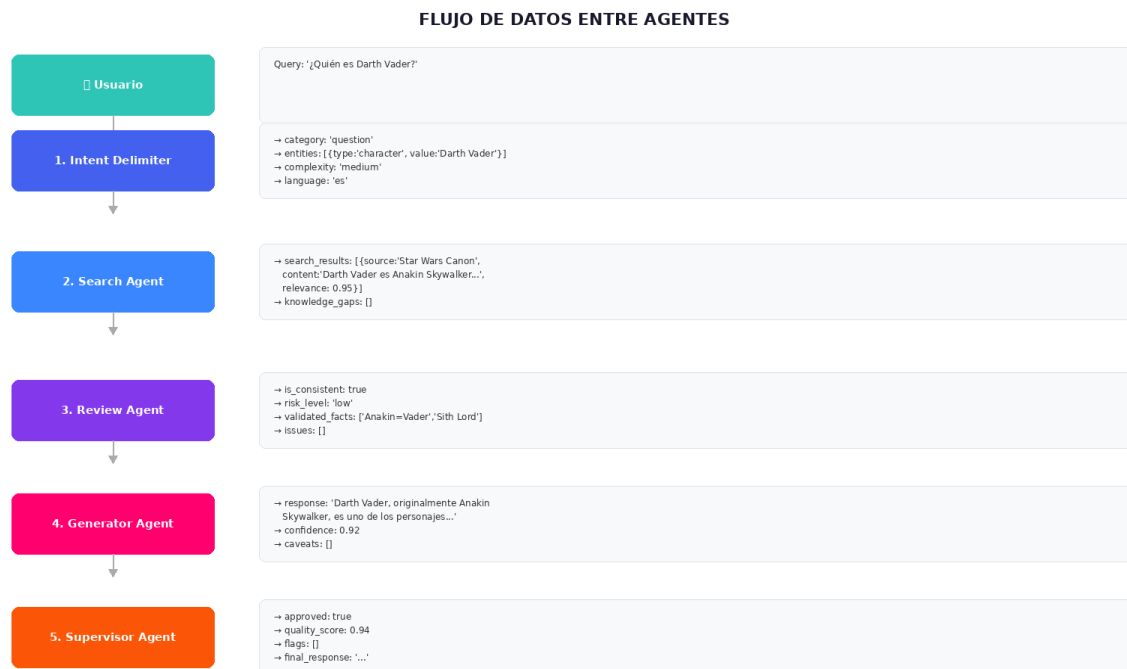


Figura 3: Flujo de datos entre agentes con ejemplo de Star Wars

Nótese que cada agente tiene acceso al output de todos los agentes anteriores, no solo del inmediatamente previo. Esto permite que el Generator, por ejemplo, considere tanto la intención original como los hechos validados por el Reviewer.

5. Stack Tecnológico

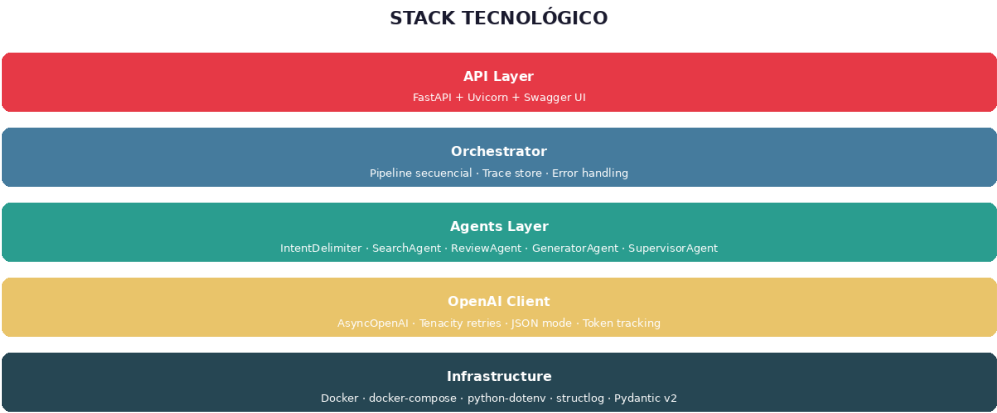


Figura 4: Capas del stack tecnológico

Capa	Tecnología	Propósito
API	FastAPI + Uvicorn	Server ASGI async con Swagger UI automático
Orquestación	Orchestrator custom	Pipeline secuencial, trace store, error handling
Agentes	5 clases Python (BaseAgent)	Herencia, system prompts, JSON structured output
LLM Client	AsyncOpenAI + Tenacity	Async, retries exponenciales, singleton pattern
Modelos	Pydantic v2	Validación de schemas, serialización JSON
Logging	structlog	Logging estructurado con contexto
Infra	Docker + docker-compose	Containerización, despliegue reproducible

6. Estructura del Proyecto

El proyecto sigue una estructura modular con separación clara de responsabilidades. La carpeta `agents/` contiene los 5 agentes, `core/` la lógica de orquestación y modelos, y `utils/` las utilidades compartidas.

ESTRUCTURA DEL PROYECTO

```
multi-agent-system/  
├── agents/  
│   ├── __init__.py  
│   ├── base.py          # Clase abstracta BaseAgent  
│   ├── intent_delimiter.py # Agente 1: Clasificador  
│   ├── search_agent.py   # Agente 2: Buscador  
│   ├── review_agent.py   # Agente 3: Revisor  
│   ├── generator_agent.py # Agente 4: Generador  
│   └── supervisor_agent.py # Agente 5: Fiscalizador  
├── core/  
│   ├── orchestrator.py   # Pipeline orchestrator  
│   ├── models.py         # Pydantic models  
│   └── config.py         # Configuración .env  
├── utils/  
│   ├── logger.py         # Structured logging  
│   └── openai_client.py  # OpenAI async wrapper  
├── main.py               # CLI entry point  
├── api.py                # FastAPI server  
├── Dockerfile  
├── docker-compose.yml  
├── requirements.txt  
└── .env
```

Figura 5: Estructura de archivos del proyecto

7. Guía Paso a Paso: Instalación y Ejecución

7.1 Prerrequisitos

- Python 3.11 o superior instalado
- Cuenta de OpenAI con billing activo y API key válida
- Docker y docker-compose (opcional, para despliegue containerizado)
- Git para clonar el repositorio

7.2 Paso 1: Clonar el Repositorio

```
git clone https://github.com/tu-usuario/multi-agent-system.git
cd multi-agent-system
```

7.3 Paso 2: Configurar Variables de Entorno

Copiar el archivo de ejemplo y editar con tu API key:

```
cp .env.example .env
```

Contenido del .env:

```
OPENAI_API_KEY=sk-proj-tu-key-aqui
OPENAI_MODEL_FAST=gpt-4o-mini
OPENAI_MODEL_SMART=gpt-4o
```

7.4 Paso 3: Instalar Dependencias

```
pip install -r requirements.txt
```

7.5 Paso 4: Verificar Conexión

Ejecutar el script de diagnóstico para verificar que la API key funciona:

```
python debug_openai.py
```

7.6 Paso 5: Ejecutar el Sistema

Opción A — CLI interactivo: `python main.py "¿Quién es más poderoso, Vader o Palpatine?"`

Opción B — API Server: `uvicorn api:app --reload --port 8000`

Opción C — Docker: docker-compose up --build**7.7 Paso 6: Probar en Swagger UI**

Abrir el navegador en <http://localhost:8000/docs> para acceder a Swagger UI. En el endpoint POST /process, hacer click en "Try it out" y enviar:

```
{  
  "query": "¿Quién es Darth Vader?",  
  "context": {"domain": "star_wars"},  
  "config": {"verbose": true}  
}
```

8. Ejemplo Completo: Star Wars Query

A continuación se muestra el recorrido completo de una query a través de los 5 agentes del pipeline, usando un ejemplo del universo Star Wars.

8.1 Input del Usuario

"¿Por qué Anakin Skywalker cayó al lado oscuro? Analiza las causas psicológicas."

8.2 Agente 1 — Intent Delimiter

Categoría: analysis

Sub-intención: Análisis psicológico de la caída de Anakin Skywalker al lado oscuro

Entidades: [{type: character, value: Anakin Skywalker}, {type: concept, value: lado oscuro}]

Complejidad: high

Confianza: 0.95

8.3 Agente 2 — Search Agent

Genera 3-4 resultados de búsqueda con información canónica sobre los factores que llevaron a Anakin al lado oscuro: miedo a la pérdida, manipulación de Palpatine, desconfianza del Consejo Jedi, trauma infantil por la esclavitud.

8.4 Agente 3 — Review Agent

Consistente: true

Riesgo: low (información canónica bien establecida)

Hechos validados: Manipulación de Palpatine, miedo a perder a Padmé, desconfianza Jedi

8.5 Agente 4 — Generator Agent

Genera un análisis psicológico completo de la caída de Anakin, estructurado con factores predisponentes (trauma infantil, apego inseguro), factores precipitantes (visiones de Padmé, exclusión del Consejo) y factores catalizadores (manipulación de Palpatine, masacre Tusken).

8.6 Agente 5 — Supervisor Agent

Aprobado: true

Quality Score: 0.94

Flags: ninguno

Veredicto: Respuesta completa, bien estructurada, basada en canon. Aprobada sin modificaciones.

9. Patrones de Diseño Aplicados

Patrón	Dónde se aplica	Beneficio
Template Method	BaseAgent abstracto	Interfaz común para todos los agentes
Singleton	OpenAIClient	Una sola conexión reutilizada
Pipeline	Orchestrator	Procesamiento secuencial ordenado
Chain of Responsibility	5 agentes en secuencia	Cada agente decide su contribución
Retry with Backoff	Tenacity en OpenAIClient	Resiliencia ante errores transitorios
Structured Output	JSON mode en todos los agentes	Parsing confiable entre agentes

10. Conclusiones y Trabajo Futuro

El sistema multi-agéntico presentado demuestra cómo la descomposición de una tarea compleja en agentes especializados produce resultados más robustos y auditables que un único prompt monolítico. La arquitectura de pipeline con fiscalización permite trazabilidad completa y control de calidad en cada etapa.

Extensiones posibles:

- Agregar agentes con herramientas externas (RAG con Wookieepedia, búsqueda web real)
- Implementar ejecución paralela de agentes independientes
- Agregar memoria persistente entre sesiones (Redis/PostgreSQL)
- Implementar feedback loop donde el Supervisor puede renviar al Generator
- Dashboard de monitoreo con métricas de tokens, latencia y calidad por agente
- Integración con frameworks como LangGraph o CrewAI para orquestación avanzada

El código fuente completo está disponible en el repositorio GitHub del curso, incluyendo Dockerfile, tests unitarios y scripts de diagnóstico.